

Hernest Koroli

Professor Reuter

SER120 Object Oriented Design

1 May 2026

Midterm #2

1. Why is an “anemic domain model” considered a criticism?

The anemic domain model is viewed negatively because it looks like objects are just storing values and don't have much functionality except for getting and setting an object's data. You can replace these objects with variables and get the same thing that the anemic domain model is doing.

2. Design an example of an anemic domain model

Cat Class

→ Methods the object can use

Cat(String name, int age, String breed) → getName(): String, getAge(): int, getBreed(): String

→ setName(newName): String, setAge(newAge): int,

setBreed(newBreed): String

Notice how the cat object doesn't really have a behavior except for storing and changing the data inside it like a data container.

3. Design an example of a rich domain model

→ Methods the object can use

A rich domain model has the storing and setting data for objects like anemic but now behavior for objects is possible with the rich domain model which I will provide.

King(String color, int moves) → move(): void, canMove(): boolean, isWhite(): boolean, isBlack(): boolean, setColor(String newColor): void, setMoves(int newMoves): void

Notice how the string has getter and setter methods, but also methods like moving and canMove() to validate if the King can move to a certain tile in a chess board. It doesn't store information, but processes it. There are methods that check a King's attributes like if it can move to a tile or whether it is too far.

4. What is the difference between your public and private key? How are they used?

A public key is another way to authenticate to do anything with your repository like pulling or committing and pushing. The private SSH key is used to "unlock" the public key and verify whether the key is from the legitimate owner. The public and private keys are essential and not having one makes the idea of accessing a repository with better security not possible.

5. What is the shell? What are your 3 favorite shell commands and why?

My favorite shell commands with the current knowledge I have with the terminal are "ls", "cd", and "nvim". I like using "ls" to check any files I have for the current directory I am in. My second favorite would be "cd" since I'm constantly changing which folders my computer is

actively “on”. Lastly, my favorite terminal or shell command is “nvim” which is not baked into the terminal since you have to install Neovim on brew for Mac. I’m trying to learn to use Neovim because I was intrigued by the shortcuts being the main point for the text-editor.

6. Quinnipiac is building a new high tech parking app using AI (QPark) which will automatically track which cars are allowed to park in which spaces and use a new 3rd party “Auto Tow” robot towing service to remove vehicles from spaces they shouldn’t be in.

1. Using Functional Decomposition break this new System down into 5-7 high level components. Defend your choices with a paragraph describing your reasoning.

Here is the [UML diagram](#) I worked on through Miro. I found the software incredibly useful for my other computer science classes and doing the Chess UML.

I added the ParkingLot class to hold important data such as the parking lot capacity and the parking lot capacity can be modified to either add or remove. Payment and Ticket are related since if a driver gets ticked for maybe illegally parking in a spot then these classes can help and have information like ticket cost or the status of whether it is paid or not. The ParkingAttendant class exists to look out for people parking in spots where it isn’t allowed legally. The ParkSpot makes each spot have attributes like whether it is occupied or not and other methods like if a vehicle can fit or not. The last is a vehicle which describes the attributes of the car like the license plate.

2. Do the same for the “Auto Tow” towing service.

I created a Student class since it's the same sort of question for Quinnipiac building a robust parking software.

Here is the [UML diagram](#) I worked on for towing.

I created the Student class and added information like name, email and the student can make payments to their tow fees. The dispatcher represents the driver towing the vehicle who can also monitor any requests or respond to emergencies like a lot of cars blocking the way. I added the vehicle class for the dispatcher to "tow" from the student and is pretty much the same with having attributes like license plate, car brand, model, etc. There is a Payment class for the student to pay any tow fee(s) he or she has. The last class is the tow truck which I put in the capacity of how many cars it can carry, the license plate, and the availability of the truck and whether it can be used for another scenario where a student could've parked illegally and needs to be towed.

7. In your opinion, does the idea of Design Patterns fit into Alan Kay's original idea of Object Oriented Programming? Write a few sentences about this.

I would say Design Patterns doesn't match with Alan Kay's idea of programming objects. Alan believed that objects should have their unique behavior. Think of the Game of Life we tried in class and how we noticed making "cells" that move on their own. Design Patterns was really addressing problems that kept appearing for programmers and showed solutions and ways of implementing and designing code to assure the code is robust and bug resistant. How would guidelines like have each class do one task or responsibility and help objects behave on their own?

8. Who are the Gang of Four?

The Gang of Four likely refers to the book called Design Patterns by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. These four authors are the “gang of four” for likely shaping how people write better code. I actually have this book before the first day of SER120 and can’t wait to read it in the summer since I have plenty of free time.

9. Given your Functional Decomposition for QPark from Question 5, create a UML diagram for the sub-system representing one of the elements.

In your design use at least one Creational, one Structural and one Behavioral Pattern

10. What is refactoring and why is it important?

Refactoring code is essentially refining existing code to either improve readability like renaming variables or changing the structure of an algorithm to make it more readable. The second benefit is you want to have a more efficient algorithm. Imagine changing from a linear search algorithm which is $O(n)$ compared to binary search which is $O(\log n)$.

11. What is the difference between “lambda” in Java and the lambda of Alonzo Church?

Lambda in java allows you to make functions without names or return type when you have a functional interface. You have to type parenthesis then make an arrow with “->” and either print something or put in any primitive or reference and these lambda function return values are inferred. There is no need to specify return like () -> “A string returned” as an example. The lambda from Alonzo Church is a mathematical expression used in calculus written as $\lambda x.M$.

The λ represents the start of the function, x represents the input, the period acts like a separator, and M acts as the function body.

12. What is the difference between an “Actor” and an “Object”?

An actor can process, store, communicate, and have “addresses”. An object in my best definition is creating a reference based on the class and using primitives and or other references to describe the attributes of an object. An object can’t process information or store information unless the user writes those methods and calls it to the object. I should also add that actors can act on their own and objects are dependent on the user calling the object’s methods.